

Symmetric Encryption: How It Works

Contents

Notation (list of symbols)	1
1 Block ciphers vs. stream ciphers	1
1.1 Symmetric encryption in one line	1
1.2 Block ciphers	2
1.3 Stream ciphers	2
1.4 Side by side	3
1.5 Symbols	3

Notation (list of symbols)

k	secret key, shared by encryptor and decryptor (symmetric).
P	plaintext: the message to encrypt, a bit string.
C	ciphertext: the encrypted output.
b	block size in bits (e.g. 128 for AES).

1 Block ciphers vs. stream ciphers

A cryptographic hash is *one-way*: you compute a digest and can never recover the message. *Encryption* is the opposite — reversible, with a secret key — and its goal is *confidentiality*: scramble a message so that only a key-holder can read it.

1.1 Symmetric encryption in one line

Symmetric means a single shared secret key k runs both directions:

$$C = \text{Enc}_k(P), \quad P = \text{Dec}_k(C).$$

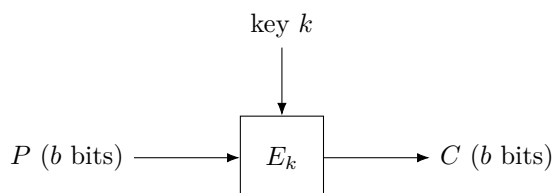
P is the *plaintext* (the message, a bit string), C the *ciphertext* (the scrambled output), and k the key. Dec_k is the exact inverse of Enc_k , so decrypting what you encrypted returns the original. Two families build Enc : *block* ciphers and *stream* ciphers.

1.2 Block ciphers

A block cipher encrypts a *fixed-size block* of bits at a time. Formally it is a keyed *permutation* — a reversible shuffle —

$$E_k : \{0, 1\}^b \rightarrow \{0, 1\}^b,$$

where b is the block size (AES uses $b = 128$ bits). “Permutation” means that for a fixed key, E_k is a bijection on the 2^b possible blocks (one-to-one and onto), so it has an inverse $D_k = E_k^{-1}$ that decryption uses. AES, and the retired DES, are block ciphers.



One block is only b bits, so to encrypt a longer message you need a *mode of operation*: a recipe for chaining many block-cipher calls (CBC, CTR, GCM, ...). Modes usually also require *padding* to fill out the final partial block. (The naive mode, ECB, encrypts each block independently and leaks repeated-block patterns — never use it.)

1.3 Stream ciphers

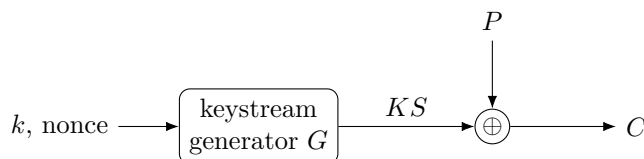
A stream cipher instead generates a long pseudorandom *keystream* from the key and a *nonce*, and combines it with the plaintext one bit (or byte) at a time with XOR:

$$KS = G(k, \text{nonce}), \quad C_i = P_i \oplus KS_i, \quad P_i = C_i \oplus KS_i.$$

The jargon:

- $\oplus$ (XOR, exclusive-or): a bit operation that outputs 1 exactly when its two input bits differ. Its key property is being self-inverse, $x \oplus y \oplus y = x$ — which is why XORing the ciphertext with the *same* keystream gives the plaintext back.
- *keystream* KS : the pseudorandom bit sequence the cipher produces to mask the message.
- *nonce* (“number used once”, a.k.a. IV): a non-secret value that makes each encryption under the same key produce a *different* keystream. It must never repeat for a given key (see the pitfall below).

This is a practical *one-time pad*: the one-time pad XORs the message with a *truly random* pad as long as the message — perfectly secret but impractical (the key is as big as the data and usable once); a stream cipher replaces that true-random pad with a pseudorandom keystream stretched from a short key. ChaCha20 is the modern choice; RC4 is the well-known broken one.



1.4 Side by side

	Block cipher	Stream cipher
Unit	fixed b -bit block	bit / byte at a time
Mechanism	keyed permutation E_k	XOR with a keystream
Long messages	need a mode (CBC, CTR, GCM)	naturally continuous
Padding	usually required	none
Examples	AES, DES (retired)	ChaCha20, RC4 (broken)
Random access	CTR yes, CBC no	yes

The line blurs. Run a block cipher in *counter* (*CTR*) mode — encrypt a running counter to make a keystream $KS_i = E_k(\text{nonce} \parallel i)$ and XOR it into the plaintext — and the block cipher *becomes* a stream cipher. So “block vs. stream” is partly about the underlying primitive and partly about how you use it.

The one fatal pitfall: never reuse a (key, nonce) pair. If two messages are XORed with the *same* keystream, then

$$C \oplus C' = (P \oplus KS) \oplus (P' \oplus KS) = P \oplus P',$$

the keystream cancels and the attacker learns $P \oplus P'$ — usually enough to unravel both messages. (This is the “two-time pad” break.) A fresh nonce per message avoids it.

1.5 Symbols

New persistent symbols this section — k, P, C, b — are in the global [list of symbols](#). The symbols below are *local to this section*.

Local symbol	Meaning (this section)
$\text{Enc}_k, \text{Dec}_k$	encrypt / decrypt under key k (exact inverses)
E_k, D_k	block-cipher permutation and its inverse, on b -bit blocks
G	keystream generator
KS	keystream (pseudorandom bits); KS_i is its i -th unit
nonce	“number used once”: non-secret, must not repeat per key
$\oplus$	bitwise XOR
P_i, C_i	i -th bit/byte of plaintext / ciphertext
i	position index